

such as multi-tasking, in which many tasks/applications can run on a single system, with the operating system allocating available resources (CPU, main memory, input/output etc) to the applications, based on the priority and criticality of each. In addition, the operating system should provide tools for application development—a Software Development Kit (SDK), and debugging, tracing and monitoring support. In short, the core objectives of any operating system are: user convenience, efficient resource allocation, and support for development and testing of applications.

Different types of kernels

An operating system mainly consists of the kernel and utilities. The core part of the operating system is the kernel, which runs directly on hardware. Its main job is to execute a given program on behalf of the user, and provide the result of execution back to the user in a human-readable format. Utilities include Graphical User Interfaces (GUIs), editors, debuggers, tracers, compilers, Web browsers and so on.

The main subsystems of a kernel are process management, file management, memory management and network management. In our next article, we will go into detail about these subsystems.

The organisation of these subsystems in the kernel leads to classification into monolithic and micro-kernels. A kernel that groups all these subsystems together in a single file is called a monolithic kernel. Most General-Purpose Operating Systems (GPOSs) like Linux, Solaris, UNIX and Windows are based on this type of kernel. GPOSs are time-sharing, and are designed to maximise throughput and provide a fair share of system resources in a multi-user/multi-process environment. Figure 1 depicts the architecture of a monolithic kernel-based operating system.

The default time-sharing system design in GPOSs is not appropriate for multimedia applications or, say, an online ticket reservation system. In such systems, critical tasks need to be assigned suitable priorities, which should persist until the task completes. The kernel should provide a facility to set a priority and scheduling policy for critical tasks—this kind of feature is referred to as ‘soft real-time’. This is achievable in GPOSs, so most of them, including Linux, are implemented with soft real-time features.

Operating systems for hard real-time applications

A ‘Hard Real Time Fatal’ system like a car Anti-lock Braking System (ABS), a flight control system, or a nuclear power plant control system needs to complete the execution of critical tasks within a specific time period. If the system misses the deadline, the results will be catastrophic (huge economic consequences or human lives at risk). The kernel should be predictable, and should have deterministic behaviour.

The monolithic GPOS design is not suitable for hard real-time systems, so we need a special-purpose, differently designed, Real-Time Operating System (RTOS)—mostly based on a micro-kernel. Benjamin Roch’s discussion on the pros and cons of the monolithic kernel versus the micro-kernel approach can be found via the *References* section [1]. In this arrangement,

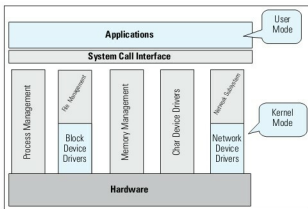


Figure 1: Architecture of a monolithic kernel-based operating system

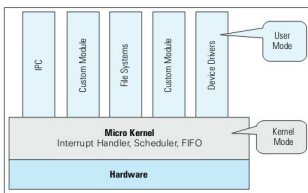


Figure 2: Architecture of a micro kernel

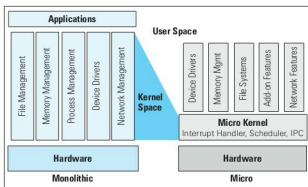


Figure 3: Comparison between monolithic and micro-kernel-based OSs

the kernel houses the essential functions to access hardware like process scheduling, interrupt handling and inter process communications. The other kernel subsystems like memory, file and network management are dynamically loaded as and when required in the user space. The demarcation between what is in the kernel and what is in user-space will differ depending on the design of the product under development, so there is no absolute specification on what constitutes the micro-kernel. Figure 2 depicts the architecture of a typical micro-kernel. Figure 3 is a comparison between monolithic and micro-kernel-based operating systems, showing the differences in what runs in kernel-space and user-space, in each type of kernel.